Report on Software Security: Lab 02 Set-User-id

Ahmed Mahfooz Ali Khan (BTHBI338)

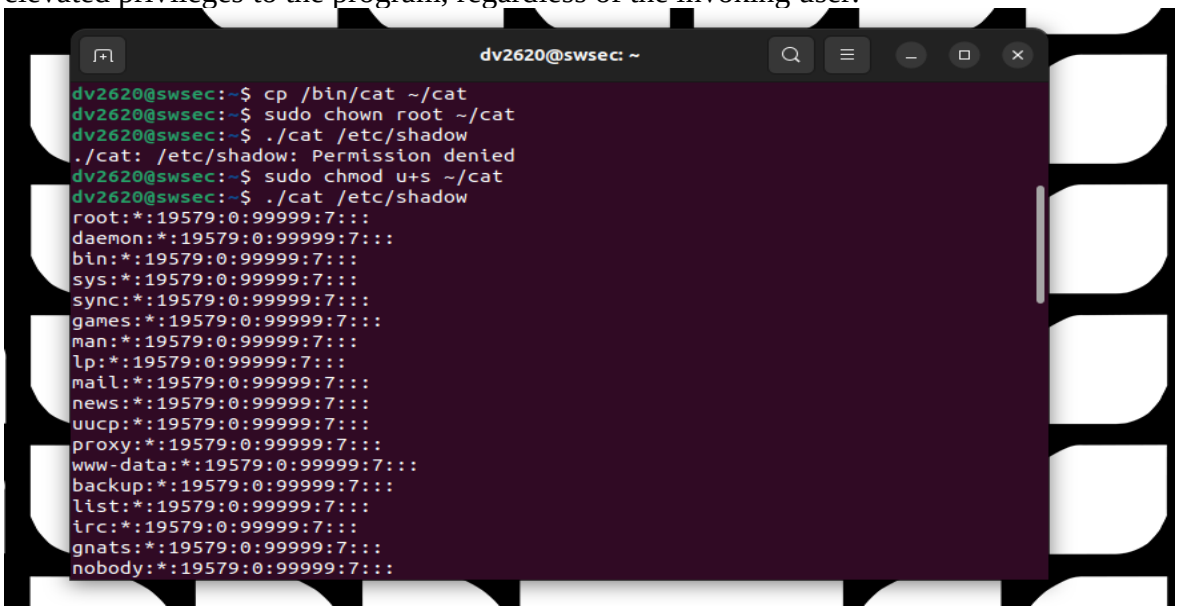
In this set of tasks, we explored various security vulnerabilities and mechanisms in Linux, specifically focusing on user privileges, the use of SUID (Set User ID), and the potential exploitation of system calls like system() and execve(). The following sections detail the outcomes of each task.

Task 1: Copying /bin/cat to Home Directory and Changing Ownership

The first task involved copying the /bin/cat program to the home directory and changing its ownership to root. After completing the task, I attempted to read /etc/shadow, a file typically owned by root. As expected, the system denied access because the current user did not have sufficient privileges to read root-owned files. This task demonstrated the importance of file ownership and access control in Linux systems, as unauthorized users cannot access sensitive files like /etc/shadow.

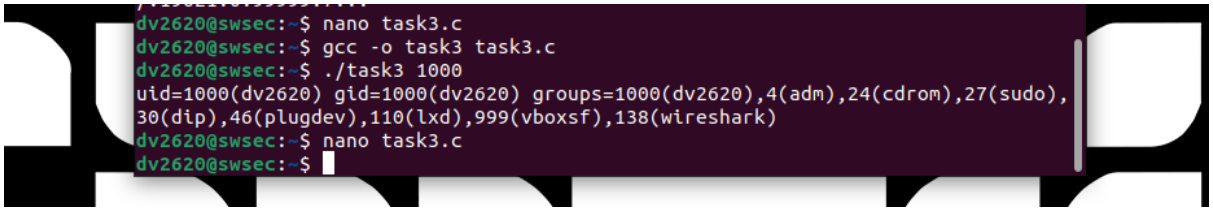
Task 2: Making /bin/cat SUID Root

The second task required setting the SUID bit on the /bin/cat program. This action allows the program to execute with the privileges of the file's owner—in this case, root. After setting the SUID bit, I attempted to use the cat program to read /etc/shadow. The result was that the program successfully executed with root privileges and displayed the contents of /etc/shadow. This task highlighted a potential vulnerability in setting SUID on executables, as it grants elevated privileges to the program, regardless of the invoking user.



Task 3: Using the system() Function to Execute Commands

In Task 3, I wrote a C program that used the system() function to execute the /bin/id command, which retrieves and prints the current user ID. The program took a user ID as input, formatted it into a shell command, and passed it to the system() function. This task demonstrated how the system() function can be used to execute external commands via the shell. While this function is convenient, it also poses a security risk if user input is not properly sanitized, as it allows for shell injection attacks.

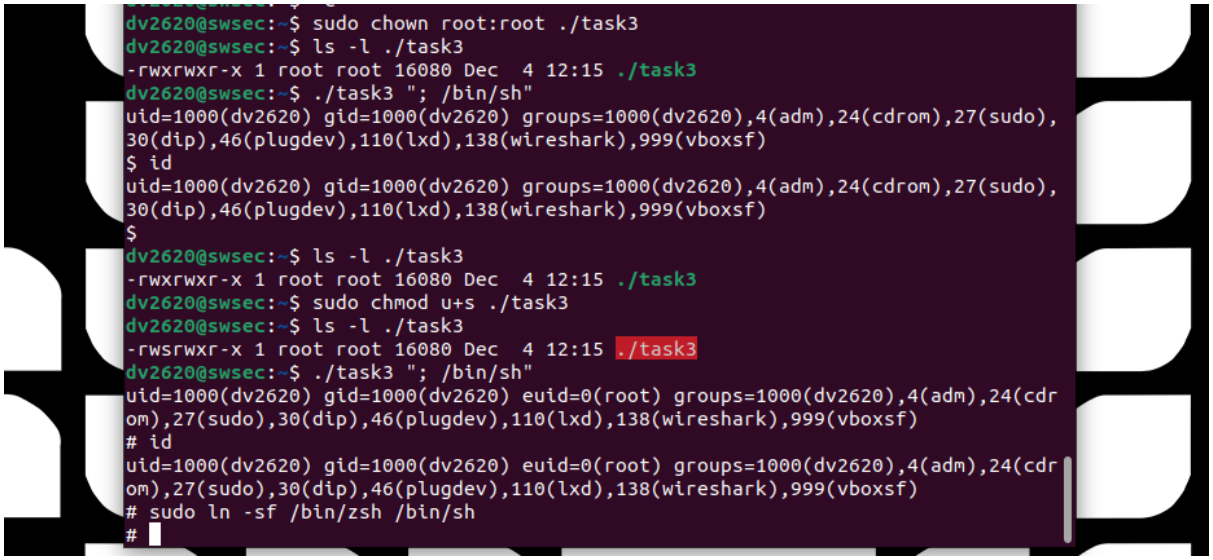


Task 4: Making the Program SUID-root

For Task 4, I set the SUID bit on the program from Task 3, which caused the program to run with root privileges. I then tested the program by passing a malicious string as input to try to spawn a shell. As a result, the program attempted to execute the malicious input as a shell command, granting root access. This task emphasized the security risks associated with using the system() function in conjunction with SUID, as unsanitized user input can lead to privilege escalation.

Task 5: Circumventing Dash Defense by Linking to /bin/zsh

In Task 5, I circumvented the dash defense mechanism (which blocks interactive shells from being executed) by creating a symbolic link from /bin/zsh to /bin/sh. This change effectively bypassed the system's security mechanism, allowing the program to execute an interactive root shell. This task demonstrated the potential for bypassing security mechanisms when administrators do not account for all possible shell interpreters.

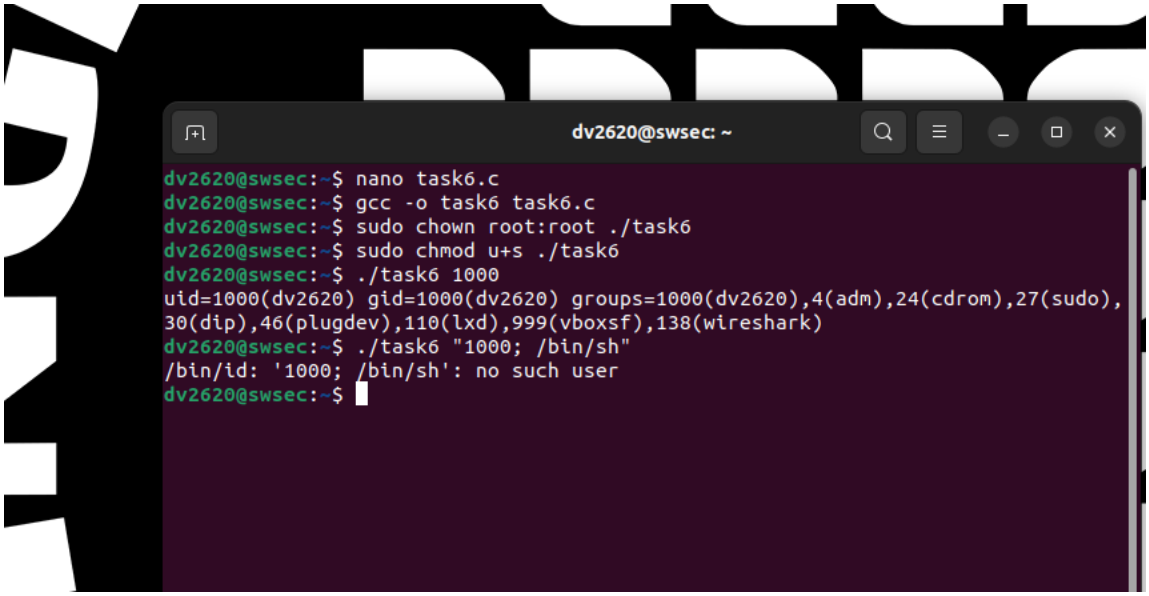


Task 6: Using execve() to Execute /bin/id

In Task 6, I wrote a C program that used the execve() system call to directly execute /bin/id, capturing and printing the output. Unlike system(), execve() does not invoke a shell, making it more secure for running commands directly. When executed, the program printed the user identity of the executing process, confirming that the execve() function was successfully used to run the command. This task showcased a more secure alternative to system(), as it avoids the shell's vulnerabilities.

Task 7: Making the execve() Program SUID-root

For Task 7, I set the SUID bit on the program from Task 6. I then tested the program by passing malicious input. However, unlike the system() function, execve() did not execute the malicious string as a shell command. This is because execve() does not invoke a shell, making it more secure and preventing shell injection attacks. The task confirmed that execve() is a safer way to execute commands in privileged programs.

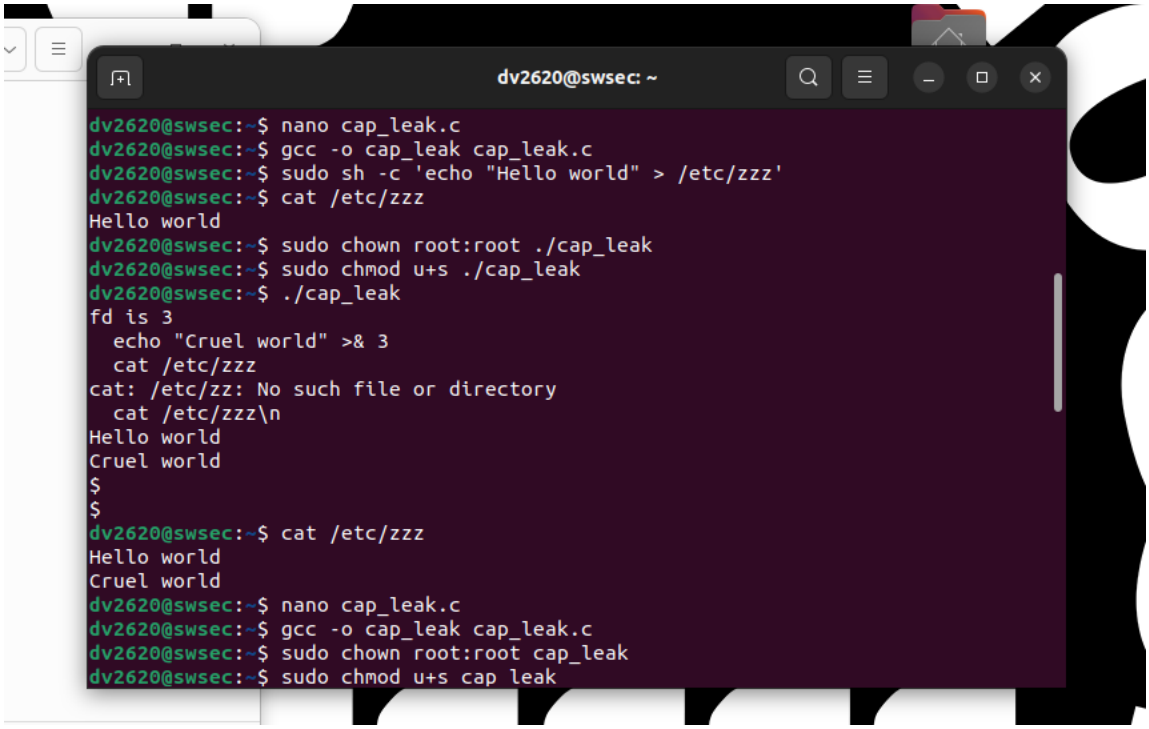


Task 8: Handling File Descriptors and Vulnerabilities

In Task 8, I compiled and ran the cap_leak.c program, which leaked a file descriptor from a privileged process to a shell. The program opened the /etc/zzz file and then executed a shell, allowing me to append text to the file through the leaked file descriptor. I successfully verified that appending data to the file was possible by redirecting output to the leaked descriptor. This task illustrated a vulnerability where sensitive file descriptors can be leaked, allowing unauthorized access to files.

Task 9: Making cap_leak SUID-root

Next, in Task 9, I set the SUID bit on the cap_leak program and tested it again. This time, running the program with SUID enabled allowed me to append text to /etc/zzz with root privileges, even though the file was not originally accessible to the user. This task highlighted the dangers of running programs with the SUID bit set, especially when they leak file descriptors or other sensitive resources.



Task 10: Fixing cap_leak Program

To mitigate the vulnerability from Task 8, I modified the cap_leak.c program to release privileged capabilities before dropping access. By doing so, I ensured that the program would no longer leak file descriptors once the privilege was revoked. After making the fix, I verified that appending to the file was no longer possible. This task demonstrated a common security measure to ensure that sensitive operations are properly handled and that privileges are carefully managed within programs.

Task 11: Restoring Default Shell

Finally, in Task 11, I restored the default shell by creating a symbolic link from /bin/sh to /bin/dash. This step ensured that the system's shell configuration was returned to its default state, undoing any changes made in previous tasks. This task concluded the series by restoring the system's integrity.

Conclusion

These tasks collectively demonstrated various security vulnerabilities in Linux systems, particularly with respect to user privileges, shell execution, and file descriptor management. By completing these tasks, I gained a deeper understanding of how privileges can be escalated and how improper handling of system calls and user input can lead to serious security issues. Additionally, I learned important techniques to secure programs, such as using execve() over system(), properly handling file descriptors, and ensuring that privileges are appropriately managed. The tasks also illustrated how attackers can exploit misconfigurations like SUID, symbolic links, and shell-based defense mechanisms to gain unauthorized access.

Deliverables:

- The source code for the C programs written in Tasks 3, 6, and 10.
- Screenshots for each task showing terminal outputs and any relevant results.

References:

class notes
chatgpt: https://chatgpt.com/share/675b586c-9840-8008-8ab3-6e43d76808f0